Constructors & Destructors

Complete Notes, Python/C++ Solutions & Object Lifecycles

Topic Overview

Types of Constructors

- **Default Constructor:** Takes no arguments (besides `self`); assigns sensible default values to attributes.
- **Parameterized Constructor:** Accepts arguments to initialize the object with custom, caller-provided values.
- **Copy Constructor:** Creates a new object by copying an existing object's data (built-in in C++; simulated in Python via class methods or copy modules).

Destructors

- **Definition:** A special cleanup method (`__del__` in Python, `~ClassName()` in C++) invoked automatically when an object is destroyed or goes out of scope.
- **Lifecycle Flow:** Object Created → Constructor Runs → Object Active → Object Destroyed → Destructor Runs.
- **Real-World Use Cases:** Auto-saving application states, closing file handles, releasing database connections.

Python Code Solutions

Task 1: Playlist – Default Constructor

```
class Playlist:
    def __init__(self):
        # Default constructor -- assigns safe default values
        self.name = "My Favourites"
        print(f"Welcome! Playlist '{self.name}' created.")

p1 = Playlist()
print("Playlist name:", p1.name)
```

Task 2: Product – Parameterized Constructor

```
class Product:
    def __init__(self, productName, price, rating):
        self.productName = productName
        self.price = price
        self.rating = rating

    def displayInfo(self):
```

```
        print(f"\nProduct: {self.productName} | Price: Rs.{self.price} |
            Rating: {self.rating}/5")

p1 = Product("Wireless Earbuds", 2499, 4.3)
p2 = Product("Phone Case", 499, 4.7)
p1.displayInfo()
p2.displayInfo()
```

Task 3: Movie – Simulated Copy Constructor

```
class Movie:
    def __init__(self, title=None, year=None):
        self.title = title
        self.year = year

    @classmethod
    def from_movie(cls, other):
        """Simulated copy constructor in Python."""
        print("(Copy constructor invoked -- duplicating movie...)")
        return cls(other.title, other.year)

    def display(self):
        print(f"Movie: {self.title} ({self.year})")

m1 = Movie("Inception", 2010)
m2 = Movie.from_movie(m1)
m1.display()
m2.display()
```

Task 4: Ticket – Destructor Lifecycle

```
class Ticket:
    def __init__(self, movieName, seat):
        print(f"Constructor: Ticket booked for '{movieName}' (Seat {seat})"
            )
        self.movieName = movieName
        self.seat = seat

    def __del__(self):
        print(f"Destructor: Releasing ticket resources... ({self.movieName
            }, Seat {self.seat})")

print("--- Program start ---")
t1 = Ticket("Avatar 3", "A12")
print("--- Ticket in use ---")
del t1
print("--- Program end ---")
```

Task 5: Playlist Auto-Save on Destruction

```
class Playlist:
    def __init__(self, name="My Favourites"):
```

```
        self.name = name
        self.songs = []

    def addSong(self, songTitle):
        self.songs.append(songTitle)

    def __del__(self):
        try:
            with open("autosave.txt", "w") as f:
                f.write(f"Playlist: {self.name}\nSongs:\n")
                for song in self.songs:
                    f.write(f" - {song}\n")
            print(f"Auto-saved '{self.name}' to disk successfully.")
        except Exception as e:
            print("Auto-save failed:", e)

p1 = Playlist("Workout Mix")
p1.addSong("Eye of the Tiger")
p1.addSong("Stronger")
del p1  # Triggers destructor auto-save
```

Summary & Key Takeaways

#	Concept	Description
1	**Default Constructor**	Initializes objects with reliable, preset baseline values without external inputs.
2	**Parameterized Constructor**	Allows flexible object creation by accepting custom data arguments at runtime.
3	**Destructor Cleanups**	Guarantees that resource cleanup (files, sockets, memory) occurs deterministically when objects expire.

Key Takeaway: Constructors guarantee that objects always start in a valid, ready state, while destructors ensure that system resources and states are safely cleaned up when objects leave scope.